

C# Entity Framework Cheatsheet

2026-05-26

Table of Contents

1. Architektur	1
2. Logging	2
2.1. Konfiguration in appsettings.json	2
2.2. Verwendung per Dependency Injection	2
2.3. Log-Level	2
3. Entity Template	4
4. Fluent API	5
4.1. Configuration Template	5
4.2. DbContext Template	5
4.3. Migration Ablauf	6
5. Repository + UnitOfWork Muster	7
5.1. Contracts in Core	7
5.2. UnitOfWork in Persistence	7
5.3. Query-Regeln für gute Read-Methoden	7
6. ReadModels / Overview-Objekte	9
6.1. Typisches Query-Template	9
7. API Design Muster	10
7.1. DTO statt Entity direkt verwenden	10
7.2. Mapper-Funktionen (DTO ↔ Entity)	10
7.3. Endpoint-Gruppe aufbauen	11
7.4. Request-Parameter in Minimal APIs	11
7.5. GET alle (GET /entities)	11
7.6. GET by id (GET /entities/{id})	12
7.7. POST create (POST /entities)	12
7.8. PUT update (PUT /entities/{id})	13
7.9. DELETE (DELETE /entities/{id})	14
8. Validierung (FluentValidation)	16
9. WebAPI Program Setup	17
10. CSV Import	19
11. Unit Tests	21
11.1. XUnit	21
11.2. NUnit	21
11.3. FluentAssertions	22
11.4. WebAPI Endpoint Tests	23
11.4.1. CustomWebApplicationFactory	23
11.4.2. Endpoint Testklasse	24

1. Architektur

Core

- Entities, Enums, Value Objects
- Interfaces (Repositories, UnitOfWork)
- QueryResult/ReadModels

Persistence

- DbContext
- Entity Configurations (Fluent API)
- Repository Implementierungen
- UnitOfWork Implementierung

WebAPI

- DTOs
- Endpoint Mapping
- Validatoren
- Dependency Injection Registrierung

Import/Console

- Datei einlesen
- Daten transformieren
- Über Repositories speichern

2. Logging

2.1. Konfiguration in appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  }
}
```

2.2. Verwendung per Dependency Injection

```
public class EntityRepository : GenericRepository<EntityName>, IEntityRepository
{
    private readonly ILogger<EntityRepository> _logger;

    public EntityRepository(
        ApplicationDbContext dbContext,
        ILogger<EntityRepository> logger) : base(dbContext)
    {
        _logger = logger;
    }

    public async Task<IList<EntityOverview>> GetOverviewAsync(string? filter)
    {
        _logger.LogInformation("Query with filter={Filter}", filter);

        // ...
    }
}
```

2.3. Log-Level

Level	Verwendung
Trace	Detaillierte Ablaufverfolgung, nur für Entwicklung
Debug	Informationen für Debugging
Information	Normale Anwendungsabläufe
Warning	Ungewöhnliche, aber behandelbare Ereignisse

Level	Verwendung
Error	Fehler, die einen Vorgang verhindern
Critical	Schwerwiegende Fehler, die sofortige Aufmerksamkeit erfordern

3. Entity Template

```
public class EntityName : EntityObject
{
    public string RequiredText { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }

    public int ParentId { get; set; }
    public ParentEntity Parent { get; set; } = null!;
}
```

4. Fluent API

Nutze Fluent API fuer DB-Regeln.

4.1. Configuration Template

```
public class EntityNameConfiguration : IEntityTypeConfiguration<EntityName>
{
    public void Configure(EntityTypeBuilder<EntityName> builder)
    {
        builder.HasKey(x => x.Id);

        builder.Property(x => x.RequiredText)
            .IsRequired()
            .HasMaxLength(100);

        builder.HasOne(x => x.Parent)
            .WithMany(p => p.Children)
            .HasForeignKey(x => x.ParentId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.HasIndex(x => x.RequiredText);
    }
}
```

4.2. DbContext Template

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options) { }

    public DbSet<EntityName> EntityNames { get; set; } = null!;
    public DbSet<ParentEntity> ParentEntities { get; set; } = null!;

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ApplicationDbContext)
.Assembly);
    }
}
```

4.3. Migration Ablauf

Entweder über die Konsole oder über: Tools > Entity Framework Core > Add Migration / Update Database

```
dotnet ef migrations add InitialCreate --project .\Persistence\Persistence.csproj
--startup-project .\WebAPI\WebAPI.csproj

dotnet ef database update --project .\Persistence\Persistence.csproj --startup-project
.\WebAPI\WebAPI.csproj
```

Im Package Manager:

```
Add-Migration InitialCreate -Project Persistence -StartupProject Import

Update-Database -Project Persistence -StartupProject Import
```

Sqlite Installation:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
# Die Design-Tools für Migrationen (Add-Migration, etc.)
5 dotnet add package Microsoft.EntityFrameworkCore.Design
```


5. Repository + UnitOfWork Muster

5.1. Contracts in Core

```
public interface IEntityRepository : IGenericRepository<EntityName>
{
    Task<IList<EntityOverview>> GetOverviewAsync(string? filter, bool? onlyActive);
}

public interface IBaseUnitOfWork : IDisposable, IAsyncDisposable
{
    Task<ITransaction> BeginTransactionAsync();
    Task<int> SaveChangesAsync();
}

public interface IUnitOfWork : IBaseUnitOfWork
{
    IEntityRepository Entities { get; }
    IAnotherRepository Others { get; }
}
```

5.2. UnitOfWork in Persistence

```
public class UnitOfWork : BaseUnitOfWork, IUnitOfWork
{
    private readonly ILogger<UnitOfWork> _logger;

    public UnitOfWork(ApplicationDbContext dbContext, ILogger<UnitOfWork> logger) :
    base(dbContext)
    {
        _logger = logger;
        _logger.LogInformation("[UnitOfWork] Initializing repositories");
        Entities = new EntityRepository(dbContext);
        Others = new AnotherRepository(dbContext);
    }

    public IEntityRepository Entities { get; }
    public IAnotherRepository Others { get; }
}
```

5.3. Query-Regeln für gute Read-Methoden

- Für reine Leseabfragen: `AsNoTracking()`
- Erst filtern, dann projizieren

- In Repository nur Datenlogik, keine API-Logik
- Rückgabe für Listen: `IList<T>` oder `List<T>` konsistent halten

6. ReadModels / Overview-Objekte

Nutze eigene Typen für Auswertungen statt Entities 1:1 nach außen zu geben.

```
public record EntityOverview(  
    int Id,  
    string DisplayName,  
    bool IsActive,  
    int RelatedCount  
);
```

6.1. Typisches Query-Template

```
public async Task<IList<EntityOverview>> GetOverviewAsync(string? filter, bool?  
onlyActive)  
{  
    _logger.LogInformation("[Repository] GetOverviewAsync called with filter={Filter},  
onlyActive={OnlyActive}", filter, onlyActive);  
    var query = _dbContext.Entities.AsNoTracking().AsQueryable();  
  
    if (!string.IsNullOrEmpty(filter))  
    {  
        _logger.LogDebug("[Repository] Applying filter: {Filter}", filter);  
        query = query.Where(x => x.Name.Contains(filter));  
    }  
  
    if (onlyActive == true)  
    {  
        _logger.LogDebug("[Repository] Filtering only active entities");  
        query = query.Where(x => x.ActiveUntil == null || x.ActiveUntil >= DateTime  
.Today);  
    }  
  
    var result = await query  
        .Select(x => new EntityOverview(  
            x.Id,  
            x.Name,  
            x.ActiveUntil == null || x.ActiveUntil >= DateTime.Today,  
            x.Children.Count))  
        .OrderBy(x => x.DisplayName)  
        .ToListAsync();  
  
    _logger.LogInformation("[Repository] GetOverviewAsync returned {Count} items",  
result.Count);  
    return result;  
}
```

7. API Design Muster

7.1. DTO statt Entity direkt verwenden

```
public record EntityDto(int Id, string Name, string Type);
```

7.2. Mapper-Funktionen (DTO $\leftrightarrow$ Entity)

Halte Mapping zentral in der Endpoint-Klasse (oder in einer separaten Mapper-Klasse), damit Create/Update/Get konsistent bleiben.

```
private static EntityName ToEntity(EntityDto dto, ILogger logger)
{
    logger.LogInformation("[Mapper] ToEntity: id={Id}, name={Name}", dto.Id, dto.
Name);
    return new EntityName
    {
        Id = dto.Id,
        Name = dto.Name,
        Type = dto.Type
    };
}

private static EntityDto? ToDto(EntityName? entity, ILogger logger)
{
    if (entity is null)
    {
        logger.LogInformation("[Mapper] ToDto: entity is null");
        return null;
    }

    logger.LogInformation("[Mapper] ToDto: id={Id}, name={Name}", entity.Id, entity
.Name);
    return new EntityDto(entity.Id, entity.Name, entity.Type);
}

private static IList<EntityDto>? ToDto(IList<EntityName>? entities, ILogger logger)
{
    if (entities is null)
    {
        logger.LogInformation("[Mapper] ToDto list: entities is null");
        return null;
    }

    logger.LogInformation("[Mapper] ToDto list: mapping {Count} entities", entities
.Count);
    return entities.Select(x => ToDto(x, logger!)).ToList();
}
```

```
}
```

Mapper-Regeln:

- `ToJson(Entity?)` gibt `null` zurueck, wenn Entity nicht gefunden wurde
- Listen immer ueber `Select(...).ToList()` abbilden
- Im `POST` und `PUT` keine Roh-Entities aus dem Request verwenden

7.3. Endpoint-Gruppe aufbauen

```
public static void MapEntityEndpoints(this IEndpointRouteBuilder app, string
baseRoute)
{
    var route = app.MapGroup(baseRoute).WithTags("Entity");

    // route.MapGet...
    // route.MapPost...
}
```

7.4. Request-Parameter in Minimal APIs

- Path-Parameter: `"/{id:int}"` + Lambda-Argument `int id`
- Query-Parameter: `[FromQuery(Name = "filter")] string? filter`
- Body-Parameter: komplexe Typen wie `EntityDto dto`
- Services per DI: z.B. `IUnitOfWork uow`

```
route.MapGet("/overview", async (
    [FromQuery(Name = "filter")] string? filter,
    [FromQuery(Name = "onlyActive")] bool? onlyActive,
    IUnitOfWork uow,
    ILogger<Program> logger) =>
{
    logger.LogInformation("[API GET /overview] filter={Filter},
onlyActive={OnlyActive}", filter, onlyActive);
    var data = await uow.Entities.GetOverviewAsync(filter, onlyActive);
    logger.LogInformation("[API GET /overview] Returning {Count} items", data.Count);
    return Results.Ok(data);
});
```

7.5. GET alle (GET /entities)

Zweck: alle Datensatze laden (read-only).

```

route.MapGet("", async (IUnitOfWork uow, ILogger<Program> logger) =>
{
    logger.LogInformation("[API GET] Fetching all entities");
    var dtos = ToDto(await uow.Entities.GetNoTrackingAsync(), logger);
    logger.LogInformation("[API GET] Returning {Count} entities", dtos?.Count ??
0);
    return Results.Ok(dtos);
})
.WithName("GetEntities")
.Produces<List<EntityDto>>(StatusCodes.Status200OK);

```

7.6. GET by id (GET /entities/{id})

Zweck: genau einen Datensatz per Path-Parameter laden.

```

route.MapGet("/{id:int}", async (int id, IUnitOfWork uow, ILogger<Program> logger) =>
{
    logger.LogInformation("[API GET /{Id}] Fetching entity", id);
    var dto = ToDto(await uow.Entities.GetByIdAsync(id), logger);

    if (dto is null)
    {
        logger.LogWarning("[API GET /{Id}] Entity not found", id);
        return Results.Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Entity not found",
            detail: $"No entity found with ID {id}");
    }

    logger.LogInformation("[API GET /{Id}] Entity found", id);
    return Results.Ok(dto);
})
.WithName("GetEntity")
.Produces<EntityDto>(StatusCodes.Status200OK)
.ProducesProblem(StatusCodes.Status404NotFound);

```

7.7. POST create (POST /entities)

Zweck: neuen Datensatz anlegen.

```

route.MapPost("", async (EntityDto dto, IUnitOfWork uow, ILogger<Program> logger) =>
{
    logger.LogInformation("[API POST] Creating entity: {@Dto}", dto);
    if (dto.Id != 0)
    {
        logger.LogWarning("[API POST] Invalid request: ID must be 0, got {Id}",

```

```

dto.Id);
    return Results.Problem(
        statusCode: StatusCodes.Status400BadRequest,
        title: "Invalid request",
        detail: "The ID in the request body must be 0");
}

using (var trans = await uow.BeginTransactionAsync())
{
    var entity = ToEntity(dto, logger);
    await uow.Entities.AddAsync(entity);
    await trans.CommitTransactionAsync();

    logger.LogInformation("[API POST] Entity created with id: {Id}", entity.
Id);
    var createdDto = ToDto(await uow.Entities.GetByIdAsync(entity.Id),
logger);
    return Results.Created($"{baseRoute}/{entity.Id}", createdDto);
}
})
.WithValidation<EntityDto>()
.WithName("AddEntity")
.Produces<EntityDto>(StatusCodes.Status201Created)
.ProducesProblem(StatusCodes.Status400BadRequest);

```

7.8. PUT update (PUT /entities/{id})

Zweck: bestehenden Datensatz ersetzen/aktualisieren.

```

route.MapPut("/{id:int}", async (int id, EntityDto dto, IUnitOfWork uow, ILogger
<Program> logger) =>
{
    logger.LogInformation("[API PUT /{Id}] Updating entity: {@Dto}", id, dto);
    if (id != dto.Id)
    {
        logger.LogWarning("[API PUT /{Id}] ID mismatch: URL={UrlId},
Body={BodyId}", id, id, dto.Id);
        return Results.Problem(
            statusCode: StatusCodes.Status400BadRequest,
            title: "Invalid request",
            detail: "The ID in the URL does not match the ID in the request
body");
    }

    using (var trans = await uow.BeginTransactionAsync())
    {
        var entity = await uow.Entities.GetByIdAsync(id);

        if (entity is null)

```

```

        {
            logger.LogWarning("[API PUT /{Id}] Entity not found", id);
            return Results.Problem(
                statusCode: StatusCodes.Status404NotFound,
                title: "Entity not found",
                detail: $"No entity found with ID {id}");
        }

        entity.Name = dto.Name;
        entity.Type = dto.Type;
        logger.LogInformation("[API PUT /{Id}] Entity updated", id);

        await trans.CommitTransactionAsync();
    }

    return Results.NoContent();
})
.WithValidation<EntityDto>()
.WithName("UpdateEntity")
.Produces(StatusCodes.Status204NoContent)
.ProducesProblem(StatusCodes.Status400BadRequest)
.ProducesProblem(StatusCodes.Status404NotFound);

```

7.9. DELETE (DELETE /entities/{id})

Zweck: Datensatz loeschen.

```

route.MapDelete("/{id:int}", async (int id, IUnitOfWork uow, ILogger<Program> logger)
=>
{
    logger.LogInformation("[API DELETE /{Id}] Deleting entity", id);
    var entity = await uow.Entities.GetByIdAsync(id);

    if (entity is null)
    {
        logger.LogWarning("[API DELETE /{Id}] Entity not found", id);
        return Results.Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Entity not found",
            detail: $"No entity found with ID {id}");
    }

    uow.Entities.Remove(entity);
    await uow.SaveChangesAsync();
    logger.LogInformation("[API DELETE /{Id}] Entity deleted", id);

    return Results.NoContent();
})
.WithName("DeleteEntity")

```



```
.Produces(StatusCodes.Status204NoContent)  
.ProducesProblem(StatusCodes.Status404NotFound);
```

8. Validierung (FluentValidation)

Halte Regeln im API-Projekt, nicht in Endpoints verstreut.

```
public class EntityDtoValidator : AbstractValidator<EntityDto>
{
    public EntityDtoValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty()
            .MinimumLength(3)
            .MaximumLength(100)
            .Matches("^[a-zA-Z0-9 ]+$")
            .WithMessage("Name can only contain letters, numbers and spaces.");

        RuleFor(x => x.Type)
            .NotEmpty();
    }
}
```

9. WebAPI Program Setup

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddProblemDetails();

builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
    {
        policy.AllowAnyHeader().AllowAnyOrigin().AllowAnyMethod();
    });
});

builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
// Falls UnitOfWork weitere Services benötigt, diese hier ebenfalls registrieren (z.B.
// IAnotherRepository)

var configuration = ConfigurationHelper.GetConfiguration();
var connectionString = configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Connection string 'DefaultConnection' not
found.");

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

builder.Services.AddValidatorsFromAssemblyContaining<Program>();

var app = builder.Build();

app.Logger.LogInformation("[Program] Application built, configuring middleware");

app.UseExceptionHandler();
app.UseStatusCodePages();
app.UseCors();

if (app.Environment.IsDevelopment())
{
    app.Logger.LogInformation("[Program] Development environment detected, enabling
Swagger");
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

// Minimal API Endpoints (Falls gewünscht, ansonsten in Extension-Methoden kapseln)
```

einfach vom Template oben nehmen und statt route direkt app verwenden

```
app.MapGet("/ping", (ILogger<Program> logger) =>
{
    logger.LogInformation("[API GET /ping] Health check called");
    return "pong";
})
.WithName("Ping")
.WithTags("Health");

// Endpoint-Mapping in Extension-Methoden kapseln, damit Program.cs sauber bleibt,
falls mehrere Entities/Endpunkt-Gruppen existieren
app.MapEntityEndpoints("/api/entity");
app.MapAnotherEntityEndpoints("/api/another-entity");

app.Run();
```

10. CSV Import

```
var builder = Host.CreateApplicationBuilder(args);

var configuration = builder.Configuration;
var connectionString = configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Connection string 'DefaultConnection' not found.");

builder.Services
    .AddDbContext<ApplicationDbContext>(options =>
        options.UseSqlServer(connectionString))
    .AddScoped<IUnitOfWork, UnitOfWork>();

var host = builder.Build();

var logger = host.Services.GetRequiredService<ILogger<Program>>();
logger.LogInformation("[Import] Migrate Database");

using (var scope = host.Services.CreateScope())
{
    var uow = scope.ServiceProvider.GetRequiredService<IUnitOfWork>();
    logger.LogInformation("[Import] Deleting database...");
    await uow.DeleteDatabaseAsync();
    logger.LogInformation("[Import] Migrating database...");
    await uow.MigrateDatabaseAsync();
    logger.LogInformation("[Import] Database migration completed");
}

logger.LogInformation("[Import] Import Data");

using (var scope = host.Services.CreateScope())
{
    var uow = scope.ServiceProvider.GetRequiredService<IUnitOfWork>();

    var csvData = await new CsvImport<TModel>().ReadAsync("ImportData/Source.csv");
    logger.LogInformation("[Import] Read {Count} rows from CSV", csvData.Count());
    var cache = (await uow.Entities.GetAsync()).ToDictionary(e => e.UniqueKey, e => e);
    logger.LogInformation("[Import] Existing entities in cache: {Count}", cache.Count);

    // LINQ-Pipeline statt foreach
    var pending = csvData.Select(async item =>
    {
        if (!cache.TryGetValue(item.Identifier, out var entity))
        {
            entity = new TEntity { PropertyA = item.ValueA, PropertyB = item.ValueB };
            await uow.Entities.AddAsync(entity);
        }
    });
}
```

```

        cache.Add(item.Identifier, entity);
        logger.LogInformation("[Import] Added new entity: {Identifier}", item
.Identifier);
    }
    entity.PropertyA = item.ValueA;
    return entity;
}).ToList(); // .ToList() erzwingt die Iteration durch die Daten

await Task.WhenAll(pending);
logger.LogInformation("[Import] Data import completed");
}

```

Falls das nicht funktioniert, hier noch eine weitere Variante:

```

using (var scope = host.Services.CreateScope())
{
    var uow = scope.ServiceProvider.GetRequiredService<IUnitOfWork>();
    var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();

    var entitiesCsv = await new CsvImport<Entity>().ReadAsync(
"ImportData/Entity.csv");
    logger.LogInformation("[Import] Read {Count} rows from CSV", entitiesCsv.Count());

    var entities = entitiesCsv.Select(x =>
    {
        return new Entity
        {
            PropertyA = x.PropertyA,
            PropertyB = x.PropertyB,
        };
    }).ToList();

    await uow.Entities.AddRangeAsync(entities);
    await uow.SaveChangesAsync();
    logger.LogInformation("[Import] Imported {Count} entities", entities.Count);
}

```

11. Unit Tests

11.1. XUnit

```
using Xunit;
using Microsoft.Extensions.Logging;
using YourProject.Services;
namespace YourProject.Tests;

public class YourServiceTests
{
    private readonly IYourService _yourService;
    private readonly ILogger<YourServiceTests> _logger = LoggerFactory.Create(b => b
.AddConsole()).CreateLogger<YourServiceTests>();

    public YourServiceTests()
    {
        _yourService = new YourService();
    }

    [Fact]
    public void YourMethod_ShouldReturnExpectedResult()
    {
        // Arrange
        var input = ...;
        _logger.LogInformation("[Test] Arrange completed");

        // Act
        var result = _yourService.YourMethod(input);
        _logger.LogInformation("[Test] Act completed, result: {Result}", result);

        // Assert
        Assert.Equal(expectedResult, result);
        _logger.LogInformation("[Test] Assert passed");
    }
}
```

11.2. NUnit

```
using NUnit.Framework;
using Microsoft.Extensions.Logging;
using YourProject.Services;
namespace YourProject.Tests;

public class YourServiceTests
{
    private IYourService _yourService;
```

```

private ILogger<YourServiceTests> _logger = null!;

[SetUp]
public void Setup()
{
    _yourService = new YourService();
    _logger = LoggerFactory.Create(b => b.AddConsole()).CreateLogger
<YourServiceTests>();
}

[Test]
public void YourMethod_ShouldReturnExpectedResult()
{
    // Arrange
    var input = ...;
    _logger.LogInformation("[Test] Arrange completed");

    // Act
    var result = _yourService.YourMethod(input);
    _logger.LogInformation("[Test] Act completed, result: {Result}", result);

    // Assert
    Assert.AreEqual(expectedResult, result);
    _logger.LogInformation("[Test] Assert passed");
}
}

```

11.3. FluentAssertions

```

using FluentAssertions;
using Xunit;
using Microsoft.Extensions.Logging;
using YourProject.Services;
namespace YourProject.Tests;

public class YourServiceTests
{
    private readonly IYourService _yourService;
    private readonly ILogger<YourServiceTests> _logger = LoggerFactory.Create(b => b
.AddConsole()).CreateLogger<YourServiceTests>();

    public YourServiceTests()
    {
        _yourService = new YourService();
    }

    [Fact]
    public void YourMethod_ShouldReturnExpectedResult()
    {

```



```

// Arrange
var input = ...;
_logger.LogInformation("[Test] Arrange completed");

// Act
var result = _yourService.YourMethod(input);
_logger.LogInformation("[Test] Act completed, result: {Result}", result);

// Assert
result.Should().Be(expectedResult);
_logger.LogInformation("[Test] Assert passed");
}
}

```

11.4. WebAPI Endpoint Tests

Nutze `WebApplicationFactory` mit `HttpClient`, um Endpoints isoliert zu testen. Datenbank und `UnitOfWork` werden durch Mocks ersetzt.

11.4.1. CustomWebApplicationFactory

```

using Core.Contracts;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.Extensions.DependencyInjection;
using NSubstitute;

namespace WebAPI.Tests;

public class CustomWebApplicationFactory : WebApplicationFactory<Program>
{
    public IUnitOfWork UnitOfWork { get; } = Substitute.For<IUnitOfWork>();

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.ConfigureServices(services =>
        {
            // Remove DbContext, UnitOfWork and all Persistence registrations
            var persistenceAssembly = typeof(Persistence.ApplicationDbContext
).Assembly;
            var descriptors = services
                .Where(d => d.ServiceType == typeof(IUnitOfWork) ||
                    (d.ServiceType.FullName != null && d.ServiceType.FullName
.Contains("DbContext"))) ||
                    (d.ImplementationType != null && d.ImplementationType
.Assembly == persistenceAssembly))
                .ToList();

            foreach (var descriptor in descriptors)

```

```

        services.Remove(descriptor);

        services.AddScoped<IUnitOfWork>(_ => UnitOfWork);
    });

    builder.UseEnvironment("Development");
}
}

```

11.4.2. Endpoint Testklasse

```

using System.Net;
using System.Net.Http.Json;
using Core.Contracts;
using Core.Entities;
using FluentAssertions;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Microsoft.Extensions.Logging;
using NSubstitute;
using WebAPI.Endpoints;

namespace WebAPI.Tests.Endpoints;

public class EntityEndpointsTests : IClassFixture<CustomWebApplicationFactory>
{
    private readonly HttpClient _client;
    private readonly IUnitOfWork _uow;
    private readonly IEntityRepository _entityRepo;
    private readonly ILogger<EntityEndpointsTests> _logger = LoggerFactory.Create(b =>
b.AddConsole()).CreateLogger<EntityEndpointsTests>();

    public EntityEndpointsTests(CustomWebApplicationFactory factory)
    {
        _client = factory.CreateClient();
        _uow = factory.UnitOfWork;
        _uow.ClearReceivedCalls();
        _entityRepo = Substitute.For<IEntityRepository>();
        _uow.Entities.Returns(_entityRepo);
        _logger.LogInformation("[EndpointTest] Setup completed, mock repository
initialized");
    }

    [Fact]
    public async Task GetEntities_ReturnsOkWithList()
    {
        var entities = new List<EntityName>
        {
            new() { Id = 1, Name = "Alpha" },
            new() { Id = 2, Name = "Beta" }
        }
    }
}

```

```

    };
    _entityRepo.GetNoTrackingAsync().ReturnsForAnyArgs(entities);
    _logger.LogInformation("[Test] GetEntities setup: {Count} entities mocked",
entities.Count);

    var response = await _client.GetAsync("/api/entity");
    _logger.LogInformation("[Test] GetEntities response: {StatusCode}", response
.StatusCode);
    var result = await response.Content.ReadFromJsonAsync<List<EntityDto>>();

    response.StatusCode.Should().Be(HttpStatusCode.OK);
    result.Should().HaveCount(2);
    result![0].Name.Should().Be("Alpha");
    _logger.LogInformation("[Test] GetEntities assertions passed");
}

[Fact]
public async Task GetEntity_ExistingId_ReturnsOk()
{
    var entity = new EntityName { Id = 1, Name = "Alpha" };
    _entityRepo.GetByIdAsync(1).ReturnsForAnyArgs(entity);
    _logger.LogInformation("[Test] GetEntity setup: entity mocked");

    var response = await _client.GetAsync("/api/entity/1");
    _logger.LogInformation("[Test] GetEntity response: {StatusCode}", response
.StatusCode);
    var result = await response.Content.ReadFromJsonAsync<EntityDto>();

    response.StatusCode.Should().Be(HttpStatusCode.OK);
    result!.Id.Should().Be(1);
    _logger.LogInformation("[Test] GetEntity assertions passed");
}

[Fact]
public async Task GetEntity_NonExistingId_Returns404()
{
    _entityRepo.GetByIdAsync(99).ReturnsForAnyArgs((EntityName?)null);
    _logger.LogInformation("[Test] GetEntity_NonExistingId setup: null mocked");

    var response = await _client.GetAsync("/api/entity/99");
    _logger.LogInformation("[Test] GetEntity_NonExistingId response:
{StatusCode}", response.StatusCode);

    response.StatusCode.Should().Be(HttpStatusCode.NotFound);
    _logger.LogInformation("[Test] GetEntity_NonExistingId assertions passed");
}

[Fact]
public async Task PostEntity_ValidDto_ReturnsCreated()
{
    var dto = new EntityDto(0, "Alpha", "TypeA");

```

```

var created = new EntityName { Id = 1, Name = "Alpha", Type = "TypeA" };
_logger.LogInformation("[Test] PostEntity setup: dto and entity mocked");

_entityRepo.AddAsync(Arg.Any<EntityName>())
    .Returns(Task.FromResult<EntityEntry<EntityName>>(null!));
_entityRepo.GetByIdAsync(Arg.Any<int>()).ReturnsForAnyArgs(created);

var response = await _client.PostAsJsonAsync("/api/entity", dto);
_logger.LogInformation("[Test] PostEntity response: {StatusCode}", response
.StatusCode);

response.StatusCode.Should().Be(HttpStatusCode.Created);
_logger.LogInformation("[Test] PostEntity assertions passed");
}

[Fact]
public async Task PostEntity_NonZeroId_ReturnsBadRequest()
{
    var dto = new EntityDto(5, "Alpha", "TypeA");
    _logger.LogInformation("[Test] PostEntity_NonZeroId setup: dto with id={Id}",
dto.Id);

    var response = await _client.PostAsJsonAsync("/api/entity", dto);
    _logger.LogInformation("[Test] PostEntity_NonZeroId response: {StatusCode}",
response.StatusCode);

    response.StatusCode.Should().Be(HttpStatusCode.BadRequest);
    _logger.LogInformation("[Test] PostEntity_NonZeroId assertions passed");
}

[Fact]
public async Task PutEntity_ValidUpdate_ReturnsNoContent()
{
    var existing = new EntityName { Id = 1, Name = "Old", Type = "TypeA" };
    var dto = new EntityDto(1, "Alpha", "TypeA");
    _entityRepo.GetByIdAsync(1).ReturnsForAnyArgs(existing);
    _logger.LogInformation("[Test] PutEntity setup: existing entity mocked");

    var response = await _client.PutAsJsonAsync("/api/entity/1", dto);
    _logger.LogInformation("[Test] PutEntity response: {StatusCode}", response
.StatusCode);

    response.StatusCode.Should().Be(HttpStatusCode.NoContent);
    _logger.LogInformation("[Test] PutEntity assertions passed");
}

[Fact]
public async Task PutEntity_IdMismatch_ReturnsBadRequest()
{
    var dto = new EntityDto(2, "Alpha", "TypeA");
    _logger.LogInformation("[Test] PutEntity_IdMismatch setup: URL id=1, body

```

```

id={BodyId}", dto.Id);

    var response = await _client.PutAsJsonAsync("/api/entity/1", dto);
    _logger.LogInformation("[Test] PutEntity_IdMismatch response: {StatusCode}",
response.StatusCode);

    response.StatusCode.Should().Be(HttpStatusCode.BadRequest);
    _logger.LogInformation("[Test] PutEntity_IdMismatch assertions passed");
}

[Fact]
public async Task PutEntity_NotFound_Returns404()
{
    var dto = new EntityDto(99, "Alpha", "TypeA");
    _entityRepo.GetByIdAsync(99).ReturnsForAnyArgs((EntityName?)null);
    _logger.LogInformation("[Test] PutEntity_NotFound setup: null mocked");

    var response = await _client.PutAsJsonAsync("/api/entity/99", dto);
    _logger.LogInformation("[Test] PutEntity_NotFound response: {StatusCode}",
response.StatusCode);

    response.StatusCode.Should().Be(HttpStatusCode.NotFound);
    _logger.LogInformation("[Test] PutEntity_NotFound assertions passed");
}

[Fact]
public async Task DeleteEntity_Existing_ReturnsNoContent()
{
    var existing = new EntityName { Id = 1, Name = "Alpha" };
    _entityRepo.GetByIdAsync(1).ReturnsForAnyArgs(existing);
    _logger.LogInformation("[Test] DeleteEntity setup: existing entity mocked");

    var response = await _client.DeleteAsync("/api/entity/1");
    _logger.LogInformation("[Test] DeleteEntity response: {StatusCode}", response
.StatusCode);

    response.StatusCode.Should().Be(HttpStatusCode.NoContent);
    _entityRepo.Received(1).Remove(existing);
    await _uow.Received(1).SaveChangesAsync();
    _logger.LogInformation("[Test] DeleteEntity assertions passed");
}

[Fact]
public async Task DeleteEntity_NotFound_Returns404()
{
    _entityRepo.GetByIdAsync(99).ReturnsForAnyArgs((EntityName?)null);
    _logger.LogInformation("[Test] DeleteEntity_NotFound setup: null mocked");

    var response = await _client.DeleteAsync("/api/entity/99");
    _logger.LogInformation("[Test] DeleteEntity_NotFound response: {StatusCode}",
response.StatusCode);

```

```
response.StatusCode.Should().Be(HttpStatusCode.NotFound);  
_logger.LogInformation("[Test] DeleteEntity_NotFound assertions passed");  
}  
}
```